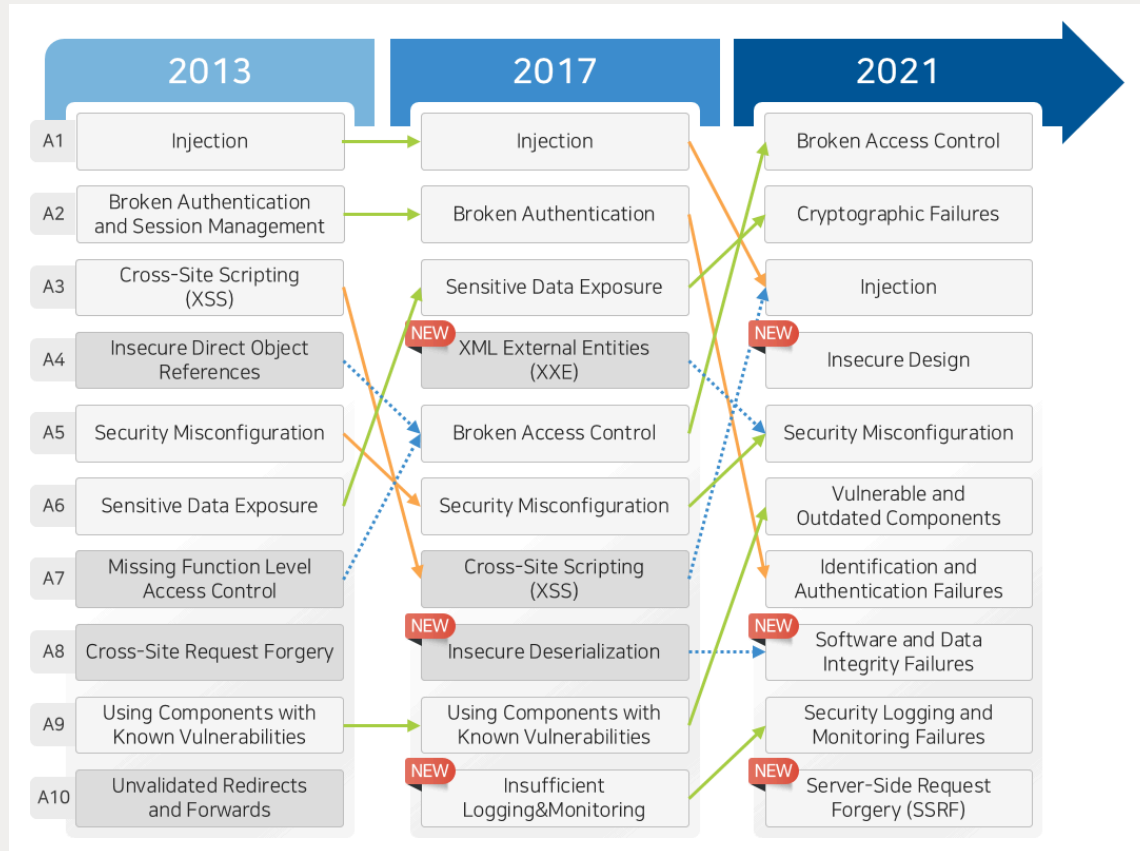




웹취약점 정리

OWASP 10



취약점 리스트	한글	의미	방안
Broken Access Control	취약한 접근 제어	인증된 사용자가 수행할 수 있는 것에 대한 제한이 제대로 적용되지 않는 것을 의미	관리자 페이지에 임의의 사용자가 접근할 수 없도록 관리자 페이지에 접근할 수 있는 권한을 가진 단말기만 접근 가능하도록 접근권한을 설정합니다
Injection (XSS 포함)	주입, 삽입	공격자가 신뢰할 수 없는 입력을 프로그램에 주입하도록하는 공격을 의미하며 이를 통해 해당 프로그램의 실행 과정의 변경을 유도	특수문자를 사용자 입력값으로 지정 금지
Security Misconfiguration (XXE 포함)	보안설정오류	최초 설치시 또는 업데이트 시 보안성을 고려하지 않은 설정을 적	애플리케이션 설치 시 기본으로 제공하는 기능, 구성 요소, 문서 및 샘플

취약점 리스트	한글	의미	방안
XML 외부 엔터티 공격 XML 입력을 구문 분석하는 응용 프로그램에 대한 공격 유형입니다. 이 공격은 외부 엔터티에 대한 참조가 포함된 XML 입력이 약하게 구성된 XML 파서에 의해 처리될 때 발생합니다.		용하는 경우가 해당된다.	을 제거한다. 신규 또는 업데이트 시에 적절한 보안 설정이 적용되었는지 검토하고, 검증하는 프로세스를 구현한다. 또한, 보안헤더를 적용해 보안을 강화한다.
Insecure Design	안전하지 않은 설계	애플리케이션의 설계 단계에 보안을 고려하지 않는 경우 발생 설계의 결함으로 인한 취약점은 이후 발견되어도 수정하기 어려움	
Software and Data Integrity Failures	소프트웨어 및 데이터 무결성 실패	무결성을 확인하지 않은 SW 업데이트로 코드/데이터 변조 막지 못함	1. 신뢰 가능한 라이브러리 사용 2. 전자서명 3. 해시
SSRF	서버 측 요청 위조	공격자가 API를 조작하여 내부 리소스 요청 시 보안 결함으로 이미지 등 데이터 노출	1. 내부 리소스 액세스 제한 ← 네트워크 세분화 2. 허용된 IP주소만 인가
Identification and Authentication Failures (Brute Force 포함)	식별 및 인증 실패	1. 다중 인증 사용X 2. 기본 계정을 사용 3. 자동화 공격이 시도 되는 경우 4. 세션 ID 재사용 등으로 인해 발생	1. 2중 인증 사용 2. 세션 ID 재사용하지 않고 폐기
Vulnerability and outdated components	취약한 컴포넌트 사용	취약한 버전이나 EoS, EoL 소프트웨어 사용시 발생	최신 버전 사용
Cryptographic Failures	암호화 실패	암호화 기능이 올바르게 구현되지 않을 때, 데이터 노출/변조 발생	평문으로 노출되지 않게 암호화한다.

ex) XML parser는 XML 문서가 XML 문법에 맞게 작성되었는지 검사

XML 문서를 읽고 해석하여 태그명, 속성명, 속성값 및 엘리먼트 내용을 분리해 주는 프로그램

OWASP 10 2021에서 유지된 것

1. Injection (XSS가 통합됨)
2. Broken Access Control
3. Security Misconfiguration (XXE가 통합됨)

OWASP 10 2021에서 새로 추가된 것

1. Insecure Design
2. Software and Data Integrity Failures
3. SSRF

OWASP 10 2021에서 이름이 바뀐 것

1. Broken Authentication → Identification and Authentication Failures
2. XXE → Security Misconfiguration
3. XSS → Injection
4. Insecure Deserialization → SW and Data Integrity Failures
5. Using Components with Known Vulnerabilities → Vulnerability and outdated components
6. Sensitive Data Exposure → Cryptographic Failures

주요정보통신기반시설 웹취약점

웹취약점 리스트 28개

Web 취약점 분석·평가 항목		
점검항목	항목 중요도	항목코드
버퍼 오버플로우	상	BO
포맷스트링	상	FS
LDAP 인젝션	상	LI
운영체제 명령 실행	상	OC
SQL 인젝션	상	SI
SSI 인젝션	상	SS
XPath 인젝션	상	XI
디렉터리 인젝션	상	DI
정보 누출	상	IL
악성 콘텐츠	상	CS
크로스사이트 스크립팅	상	XS
약한 문자열 강도	상	BF
불충분한 인증	상	IA
취약한 패스워드 복구	상	PR
크로스사이트 리퀘스트 변조(CSRF)	상	CF
세션 예측	상	SE
불충분한 인가	상	IN
불충분한 세션 만료	상	SC
세션 고정	상	SF
자동화 공격	상	AU
프로세스 검증 누락	상	PV
파일 업로드	상	FU
파일 다운로드	상	FD
관리자 페이지 노출	상	AE
경로 추적	상	PT
위치 공개	상	PL
데이터 광문 전송	상	SN
쿠키 변조	상	CC

웹취약점

2. 필요한 크기만큼 파라미터 값을 받을 수 있도록 제한

2. 포맷 스트링

보안 위협 C언어로 만드는 프로그램 중 변수의 값을 입출력 받을 때, 그 값을 조작하여 프로그램의 메모리 위치를 반환받아 **메모리 주소**를 **변조**하여 시스템의 **관리자 권한**을 **획득**

포맷 스트링 버그 printf등 함수에서 **문자열 입력 포맷**을 잘못된 형태로 입력하는 경우 나타나는 취약점으로 루트 권한을 획득하는 것도 가능함.

%n은 이전까지 입력되었던 문자열의 길이 만큼 해당 변수에 저장시키기 때문에 메모리의 내용도 변조 가능하므로 포맷 스트링 취약점의 핵심이기도 함.

점검 대상 웹 앱 소스코드, C/S(Client-Server) 프로그램

판단 기준

- 양호 : 포맷 스트링 버그를 발생시키는 문자열 입력 시 검증 로직이 존재하여 오류가 발생하지 않는 경우
- 취약 : 포맷 스트링 버그를 발생시키는 문자열 입력 시 검증 로직이 미흡하여 오류가 발생하는 경우

조치 방법

1. 웹 서버 프로그램을 최신 버전 업데이트
2. 포맷 스트링 버그 발생 시키는 문자열에 대한 검증 로직 구현

3. LDAP 인젝션

보안 위협 로컬 프록시 사용해 LDAP문 변조

대응 방안 사용자 입력 값으로 영문, 숫자만 허용

판단 기준

- 양호 : LDAP 쿼리 입력에 대한 검증이 이뤄져 변조 쿼리 실행되지 않는 경우

4. 운영체제 명령 실행 (OS 명령 실행)

점검 목적 적절한 검증절차를 거치지 않은 사용자 입력 값에 의해 의도치 않은 시스템 명령어가 실행되는 것을 방지

보안 위협 | & 문자에 대한 필터링 구현

- `|` = 첫번째 명령어가 성공했는지 여부 상관 없이 두번째 명령어 실행
- `&` = 윈도우 해석기에서 첫 번째 명령이 성공 했을 경우 두번째 명령어 실행
- `'` = `'ls -al'` 같이 역 작은따옴표 내 명령어를 먼저 실행

조치 방안 웹/앱 서버 최신 버전 업데이트 적용, 앱은 운영체제로부터 직접적으로 명령어 호출하지 않도록 구현

5. SQL Injection

점검 목적 대화형 웹 사이트에 비정상적 사용자 입력 값 허용을 차단하여 악의적 DB 접근 조작 방지

보안위협 비정상적인 SQL 쿼리로 DBMS 및 데이터를 열람하거나 조작 가능하므로 사용자의 입력 값에 대한 필터링 구현

조치 방안 서버에 검증되지 않는 SQL 쿼리 요청 시 에러 페이지가 아닌 정상 페이지가 반환 되도록 필터링 처리하고 웹 방화벽에 SQL 인젝션 관련 룰셋을 적용하여 SQL 인젝션 차단

점검 방법

1. SQL 쿼리 삽입하여 DB 에러 페이지가 반환되는지 확인
2. SQL 참, 거짓 쿼리 삽입하여 참, 거짓에 따라 반환되는 페이지가 다른지 확인
3. 로그인 페이지에 참이 되는 SQL 쿼리를 전달하여 로그인되는지 확인

보안 설정 방법

1. 문자열 유효성 검증하는 로직 구현
 2. `'` `;` `--` `#` `/* */` 등 사용자 입력값으로 금지
 3. Dynamic SQL 구문 사용 지양 및 파라미터 검사 적용
1. 특수문자를 hex문자(0x00)로 바꿔준다.
 2. 선처리 질의문 (prepared statement) (쿼리구조/파라미터는 변경되지 않도록 설정 하고, 입력값만 받는다.)

6. SSI (Server Side Inclusion) 인젝션

점검 목적 적절한 입력 값 검증 절차를 마련하여 악의적인 파일을 include 시키지 못하도록 하여, 불법적인 데이터 접근 차단

보안 위협 웹 서버 상 파일을 include 시켜 명령문이 실행 되게 하여 웹 서버의 CGI 환경 변수를 설정하고 출력하는 등 데이터 접근 가능하게 함.

SSI 인젝션 HTML 문서 내 입력받은 변수 값을 서버 측에서 처리할 때 부적절한 명령문이 포함/실행 되어 서버의 데이터가 유출되는 취약점

대응 방안

소스코드 및 취약점 점검 필요

특수문자에 대한 **Entity** 형태로 표시

변경 전	<	>	"	(	)	#	&
변경 후	<	>	"	(	)	#	&

점검 방법

1. `<!--#echo var="DOCUMENT_ROOT" -->` 삽입해 전송 후 반환되는 페이지에 사이트의 홈 디렉터리가 표시되는지 확인
2. HTTP 요청 헤더에 명령어를 삽입하여 실행되는지 확인

GET / HTTP/1.0

Referer: `<!--#exec cmd="/bin/ps ax" -->`

User-Agent: `<!--#include virtual="/proc/version" -->`

7. XPath 인젝션

보안 위협 프로그래머가 **의도하지 않았던 문자열을 전달**하여 쿼리문의 **의미를 왜곡**시킴

점검 방법

- `'and'a='a` `'and'a='b` 삽입하여 쿼리의 참 거짓에 따라 반환되는 페이지가 다른지 확인

보안 설정

`[]` `:` `/` 등 XPath 쿼리 파괴하는 특수문자는 입력하지 못하게 하여야 함

XPath란? XPath는 XML문장 속의 요소, 속성등을 **설정**하기 위한 언어이다.

8. 디렉터리 인덱싱

보안 위협 브라우저를 통해 특정 디렉터리 내 파일 리스트를 노출하여 응용시스템(워드, 엑셀 등)의 구조를 외부에 허용

점검 방법 URL 경로중 확인하고자 하는 디렉터리까지만 주소창에 입력하여 인덱싱 여부 확인

보안 설정 웹 서버 환경설정에서 디렉터리 인덱싱 기능 제거

BEFORE

```
<Directory "/var/www/html">
Options Indexes
</Directory>
```

AFTER

```
<Directory "/var/www/html">
Options
</Directory>
```

WINDOWS

- IIS 7.0에서 **디렉터리 검색** 체크 해제

크롤링과 인덱싱

크롤러는 공개적으로 접근 가능한 웹페이지를 방문하며, 공개 페이지에 있는 링크를 따라간다. 그리고 페이지에서 찾은 정보와 콘텐츠를 검색 색인에 저장한다. 이 때, 페이지의 링크를 따라다니며 정보를 수집하는 것을 크롤링, 페이지에 찾은 정보를 분석해 색인하는 것을 인덱싱이라고 한다. robots.txt를 사용해 크롤링을 제어하거나, sitemap.xml을 통해 사이트 URL 정보나 업데이트 빈도 등을 크롤러에게 알려줄 수 있다.

robots.txt 크롤링 제어

이전 과제에서 robots.txt에 대해서 알아보았으므로, 경우의 수에 따라 크롤러 제어하는 방법을 알아보려고 한다.

- 모든 문서 차단하는 경우

```
User-agent: *  
Disallow: /
```

- 모든 문서가 허용되는 경우

```
User-agent: *  
Disallow:
```

- 특정 폴더 차단하는 경우

```
User-agent: *  
Disallow: /users/  
Disallow: /admin/
```

- 특정 엔진만 차단하는 경우

구분	경로 추적(Path Traversal)	디렉터리 인덱싱(Directory Indexing)
목적	상위 디렉터리로 이동하여 민감한 파일 접근	특정 디렉터리 내 파일 목록을 노출
공격 방식	../ 또는 절대 경로를 사용하여 파일 직접 접근	웹 서버 설정 미스로 파일 리스트가 노출됨
위험 요소	패스워드, 설정 파일 등의 직접 유출	백업 파일, 중요 문서 등이 목록에서 노출
대응 방법	입력 검증, 경로 정규화, 접근 제한	디렉터리 인덱싱 비활성화, 인덱스 파일 추가

9. 정보 누출

보안 위협 웹 사이트에 중요 정보(개인정보, 계정정보, 금융정보)가 노출되거나 에러 발생 시 과도한 정보(DB, 웹서버 정보)가 노출되어 공격자들이 2차 공격을 위한 정보로 활용할 수 있음.

점검 방법

1. 웹 사이트에 중요정보가 평문으로 노출되고 있는지 확인 e.g. 비밀번호
2. 웹페이지에 **마스킹** 된 중요 정보(비밀번호 ●●●●● 으로 처리)가 웹페이지 소스에 평문으로 노출되고 있는지 확인
3. 에러 메시지/페이지에 과도한 정보가 노출되는지
4. 인코딩된 중요 정보가 디코딩이 되는지

조치 방안

Windows

- IIS > [사용자 지정 오류] > 400~500등 에러 페이지 별도 지정
 - 403 Forbidden은 금지 되었다는 정보가 비인가자에게 주어지기 때문에, 404 Not Found로 변경해야함
 - 경로가 나와 있거나, DB에 대한 정보가 노출되는 에러 페이지도 있어 사용자가 별도로 에러 페이지를 지정해야함

10. 악성 콘텐츠 (파일 업로드와 비슷)

보안 위협 웹 사이트 게시판, 댓글, 자료실 등에 정상적인 콘텐츠 대신 악성 콘텐츠를 주입하여 실행될 경우 사용자가 해당 콘텐츠 열람 시 악성코드 감염 및 웹페이지 변조 등 보안상 심각한 위험에 노출

점검 방법

1. **콘텐츠 삽입** 및 **파일 업로드** 제한 **필터링** 적용 여부 점검
2. 게시판 등 페이지에서 강제적으로 이뤄지는 악의적인 프로그램 다운로드 및 콘텐츠 자동 실행이나 악의적인 사이트로의 이동이 발생하는지 확인

보안설정 방법

1. 주기적으로 **업로드된 파일** 을 대상으로 **바이러스 검사** 실시
2. 악성 콘텐츠 혹은 악성 콘텐츠가 삽입된 **페이지 삭제**

11. 크로스 사이트 스크립팅 (XSS)

보안위협 웹 앱에서 사용자 입력 값에 대한 필터링이 제대로 이뤄지지 않는 경우, 공격자가 사용자 입력 값을 받는 **게시판**, **URL** 등에 **악의적인 스크립트(JS)** 삽입하여 게시글이나 이메일을 읽는 **사용자의 쿠키(세션)**을 탈취하여 도용하거나 **악성코드 유포 사이트로 Redirect** 할 수 있음

조치 방법

1. 웹 사이트의 게시판, 1:1 문의, URL 등에서 사용자 입력 값에 대해 검증 로직을 추가
2. **HTML 코드** 중 **필요한 코드** 만 입력 되게 설정
3. 1. 특수문자를 html entity(&#xxx;) 형태로 바꾼다. htmlspecialchars()함수 사용해 바꿈.
2. **웹프락시** 우회 가능한 서버를 검증한다.
3. 화이트리스트 방식 필터링

12. 약한 문자열 강도

보안 위협 **유추** 가 **용이** 한 **계정** 및 **패스워드** 의 사용으로 인한 사용자 권한 탈취 위험이 존재하며, 해당 위험을 방지하기 위해 값의 **적절성** 및 **복잡성** 을 검증하는 로직을 구현해야 함

13. 불충분한 인증

보안위협 중요 정보(개인정보 변경 등) 페이지에 대한 인증 절차가 불충분할 경우 권한이 없는 사용자가 중요정보 페이지에 접근하여 정보를 유출하거나 변조할 수 있으므로 OTP 등 추가적 인증 절차 필요

14. 취약한 패스워드 복구

보안위협 취약한 패스워드 찾기 등으로 공격자가 불법적으로 다른 사용자의 패스워드를 획득/변경

점검 방법 재설정 되는 패스워드 몇개를 획득하여 사용자의 연락처/주소/메일을 패스워드로 이용하고 있는지 확인하고 재설정된 패스워드를 인증된 사용자 메일/SMS로 전송하고 있는지 확인

보안설정 개인정보로 패스워드를 생성하지 말아야 하며, 난수를 이용한 불규칙적이고 최소 길이 6자 이상의 패턴이 없는 패스워드를 발급받아야 함.

15. 크로스사이트 리퀘스트 변조 (CSRF)

보안 위협 사용자의 인증 정보 내에서 사용자의 요청을 변조함으로써 해당 사용자의 권한으로 악의적인 공격을 수행할 수 있음

판단기준

- 취약 : 사용자 입력 값에 대한 필터링이 이뤄지지 않으며, HTML 코드를 입력하여 실행 되는 경우

점검 방법

1. XSS 취약점이 존재하는지 확인
2. 등록 및 변경 등의 데이터 수정 기능이 페이지에 있는지 조사
3. 임의의 명령을 수행하는 스크립트 삽입 후 해당 게시글을 타 사용자가 열람하였을 경우 스크립트가 실행되는지 확인

보안 설정

1. HTML / JS 태그 사용을 사전에 제한
2. 웹 사이트에 사용자 입력 값이 저장되는 페이지는 요청이 일회성 이 될 수 있게 설계

16. 세션 예측

보안 위협 사용자에게 전달하는 세션ID 가 일정한 패턴 을 가지고 있는 경우 공격자가 세션 ID를 추측하여 불법적인 접근을 시도할 수 있음

판단기준

- 양호 : 추측 불가능한 세션 ID가 발급되는 경우
- 취약 : 세션 ID가 일정한 패턴으로 발급되는 경우

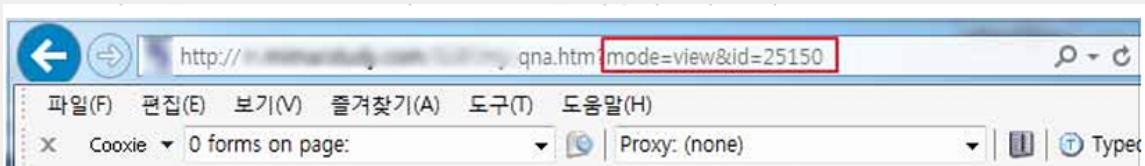
17. 불충분한 인가

보안 위협 접근제어가 필요한 중요 페이지의 통제수단이 미흡한 경우, 비인가자가 URL 파라미터 값 변경 등의 방법으로 중요 페이지에 접근하여 민감한 정보 열람 및 변조 가능

조치 방법 접근제어가 필요한 모든 페이지에 **권한검증 로직** 구현

점검 방법

1. 비밀 게시물/개인정보 변경/패스워드 변경 페이지에서 다른 사용자와의 구분을 ID, 일련번호 등의 단순한 값을 사용하는지 조사



2. 게시글을 구분하는 파라미터 값을 변경하는 것만으로 다른 사용자의 비밀 게시물/개인정보 변경/패스워드 변경 페이지에 접근 가능한지 확인



18. 불필요한 세션 만료

보안설정 타임아웃 시간을 10분으로 설정

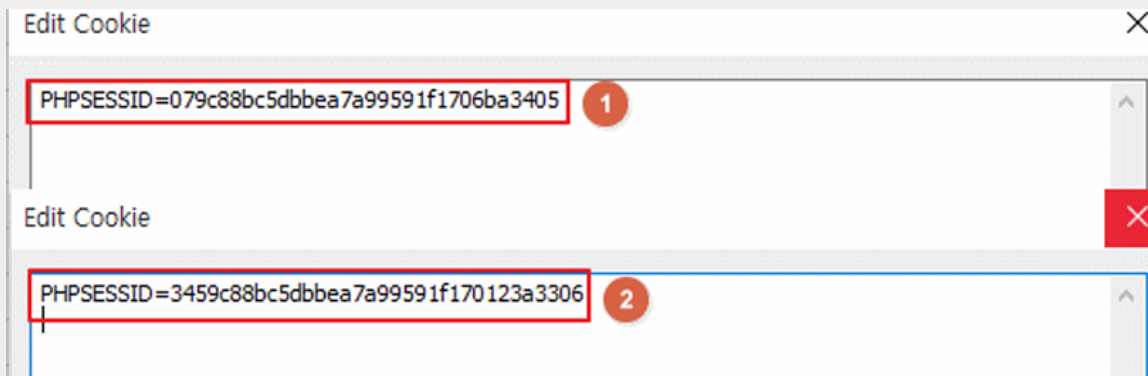
19. 세션 고정

보안 위협 사용자 로그인 시 항상 일정하게 고정된 세션 ID가 발행되는 경우 세션 ID를 도용한 비인가자의 접근 및 권한 우회가 가능

조치 방법 사용자가 로그인할 때마다 예측 불가능한 새로운 세션 ID 생성 로직 구현하고 기존 세션 ID를 파기

점검 방법

로그인 시 세션 ID가 발행되는지 확인 → 로그아웃 → 재로그인시 예측 불가능한 새로운 세션 ID 발급되는지 확인



20. 자동화 공격

보안 위협 웹 앱의 특정 프로세스에 대한 반복적인 요청을 통제하지 않을 경우 무차별 대입 공격으로 인해 사용자 계정을 탈취할 수 있고, 자동화 공격으로 게시글 등록 또는 SMS 발송 요청을 반복하여 웹 앱 자원을 고갈시킬 수 있음

판단 기준

- 취약 : 웹 앱의 특정 프로세스에 대한 반복적인 요청 시 통제가 미흡한 경우

보안 설정 사용자 요청이 일회성이 될 수 있도록, CHAPCHA 등을 로직으로 이용.

21. 프로세스 검증 누락

보안 위협 인증이 필요한 웹 사이트의 중요 페이지(관리 페이지 등)에 대한 접근 제어 가 미흡
할 경우 하위 URL 직접 접근, 스크립트 조작 등의 방법으로 중요한 페이지에 대한 접근이 가능함

판단 기준

- 취약 : 웹 사이트의 하위 URL을 로그인하지 않고 직접 접근할 때 접근이 가능한 경우

보안설정 우회 될 수 될 수 있는 플로우를 차단

22. 파일 업로드

보안 위협 공격자는 **조작된 Server Side Script 파일** 을 **서버**에 업로드 및 실행하여, 시스템 **관리자 권한 획득** 또는 **인접 서버**에 대한 **침입**을 시도

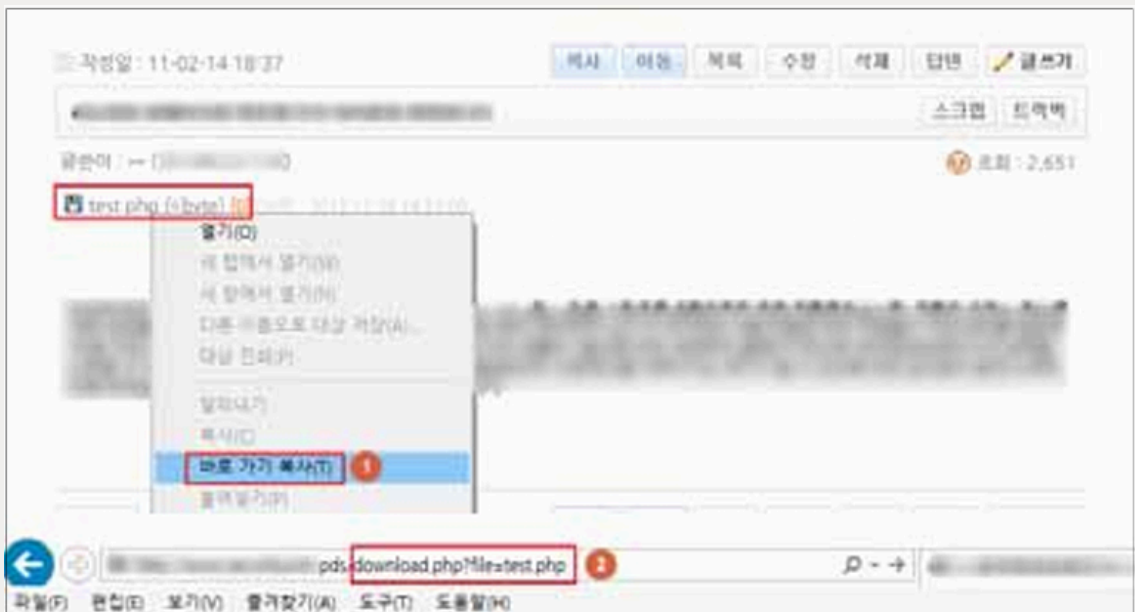
판단 기준

- 취약 : 업로드 되는 파일에 대한 **확장자 검증**이 이루어지지 않는 경우

조치 방법 업로드 되는 파일에 대한 확장자 검증 및 실행 권한 제거

점검 방법

- 웹 사이트에 파일 업로드 기능이 존재하는 경우, 확장자가 jsp, php, asp, cgi 등 server side script 파일들이 업로드 가능한지 확인
- 브라우저 URL에서도 Server side script 파일의 위치를 조사한 후, 브라우저 주소창에 해당 경로를 입력하여 실행 가능한지 확인



보안 설정

- 화이트 리스트 방식으로 허용된 확장자만 업로드 가능토록 서버 측 통제 적용
- 외부 사용자가 추측할 수 없는 문자열로 확장자를 변경하여 저장

Windows

IIS > 홈 디렉터리 > 해당 업로드 폴더 우클릭 > 실행 권한 > 없음

23. 파일 다운로드

보안 위협 공격자가 파일 다운로드 시 앱의 파라미터 값을 조작하여 웹 사이트의 중요한 파일(DB, 앱 파일)을 다운받을 수 있음

판단 기준 다운로드 파일이 저장된 디렉터리 이외에 접근이 가능한 경우

점검 방법

1. 웹 사이트에 cgi, jsp, php 등 앱 이용하여 파일을 다운받는 페이지가 있는지 조사
2. 상대 경로(../) 치환하여 전송 시 해당 경로 파일들을 다운로드 할 수 있는지 확인
3. 다운로드가 불가능한 경우 변조한 파일 경로를 아래의 인코딩을 적용하여 재전송 후 다운로드 가능한지 확인

24. 관리자 페이지 노출

보안 위협 웹 관리자의 권한이 노출될 경우, 웹 사이트의 **변조** 뿐 아니라 취약성 정도에 따라서 웹 서버의 **권한** 까지도 **노출** 될 수 있음

판단 기준

- 취약 : 유추하기 쉬운 **URL**로 **관리자** 페이지 **접근** 이 가능한 경우

조치 방법 유추하기 어려운 이름(포트 번호 변경 포함)으로 관리자 페이지를 변경하여 **비인가자** 가 관리자 페이지에 접근할 수 없도록 한다.

점검 방법

1. 추측하기 쉬운 관리자 페이지 경로 (/admin, /manager, /master, /system) 접근 시도하여 관리자 페이지가 노출되는지 확인
2. 추측하기 쉬운 포트 (8888 등) 접속을 시도하여 관리자 페이지가 노출되는지 확인
3. 관리자 페이지 로그인 창에 기본 관리자 계정(admin, manager 등) 및 기본 패스워드를 입력하여 로그인 가능한지 확인

25. 경로 추적 (Path Traversal)

보안 위협 웹 서버/ 웹 앱의 파일 또는 디렉터리 접근이 통제되지 않아, **중요한 파일** 과 데이터에 접근을 허용 하는 취약점.

웹 루트 디렉터리에서 외부의 파일까지 접근하여 이를 실행할 수 있음

판단 기준

- 취약 : 웹 루트 디렉터리보다 상위 디렉터리(/root)에 접근이 가능한 경우

점검 방법 ../../etc/passwd 등 경로 변조 후 전송하여 파일 내용이 웹 브라우저에 표시되는지 확인

보안 설정 접근하려는 파일이 있는 디렉터리에 **chroot 환경** 적용 시 경로 추적 공격을 최소화
Windows

IIS > 해당 웹 사이트 > 속성 > 홈 디렉터리 > 구성 > 옵션 > [부모 경로 사용] 체크 해제

26. 위치 공개

보안 위협 폴더나 파일명의 위치가 예측 가능하여 쉽게 노출될 경우 공격자는 이를 악용하여 대상에 대한 정보를 획득하고 민감한 데이터에 접근 가능

판단 기준

- 취약 : 불필요한 파일이 존재하거나, 샘플 페이지가 존재하는 경우

점검 방법

- 웹 루트 디렉터리 내 웹 서비스에 불필요한 확장자 (.bak, .org, .tmp) 파일이 존재하는지 확인
- 각종 샘플페이지(cgi-bin, manual, usage)의 디렉터리 및 파일 존재 여부 확인

보안설정 robots.txt 파일 작성을 통해 검색 차단할 디렉터리, 확장자, 페이지 등을 지정

27. 데이터 평문 전송

보안 위협 웹상의 데이터 통신은 대부분 텍스트 기반으로 이뤄지기 때문에 서버와 클라이언트 간에 암호화 프로세스를 구현하지 않으면 간단한 도청을 통해 정보 탈취 됨

조치 방법 중요정보 전송구간(로그인, 회원가입, 게시판) 암호화 통신 (https) 적용

점검 방법 1. 로그인 페이지 확인 2. 암호화 통신 여부 확인 3. 취약한 버전 SSL 2.0 미만 사용하는지 확인

보안 설정 TLSv1.2 이상 사용 권장, 웹에 개인정보 최소화

28. 쿠키 변조

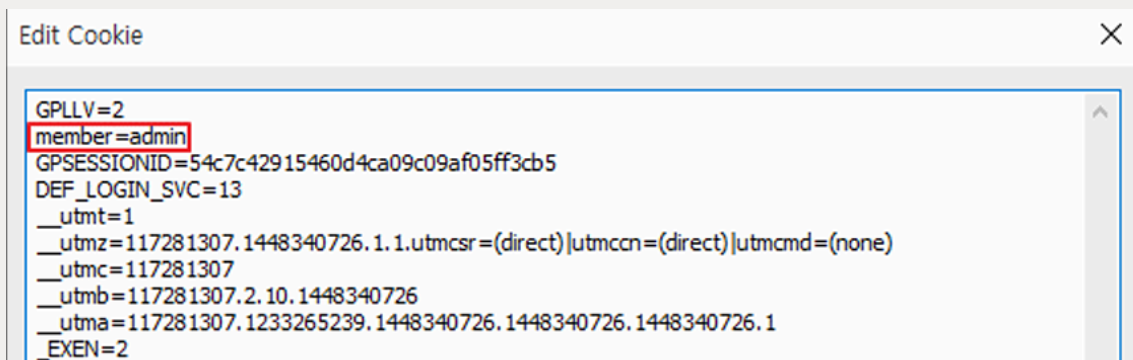
보안 위협 클라이언트에 전달되는 쿠키에 사용자 식별 값이 평문으로 노출될 경우, 쿠키 변조를 통해 **다른 사용자의 유효한 세션** 을 **취득** 할 수 있어, 기타 중요 정보의 유출/변조 가능

판단기준

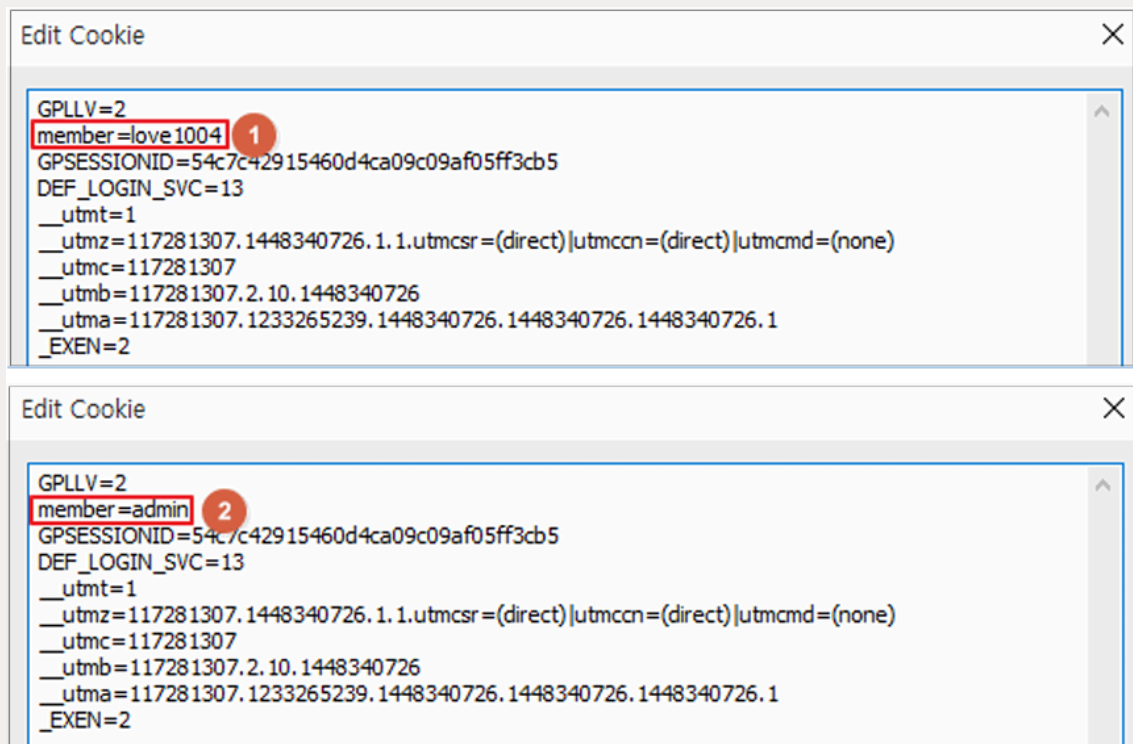
- 취약 : 안전한 알고리즘이 적용되지 않은 쿠키/세션을 사용하거나, **client side session** 을 사용하는 경우

점검 방법

1. 쿠키 내용에 중요 정보 노출 조사



2. 쿠키의 중요정보 변경하여 다른 사용자 및 권한으로 정상 이용 가능한지 확인



보안 설정 SEED, 3DES, AES 같은 안전한 알고리즘 사용, Server Side Session 방식 이용,

Set-Cookie : secure, HttpOnly

등 사용

빈출

인증	인가
사용자가 맞는지 검증	사용자가 권한이 있는지 검증

침투	해킹
특정 시스템이나 네트워크에 초점 맞추어 상세하게 취약점을 식별/분석	조직 전체의 보안 체계와 다양한 위협 시나리오를 전반적으로 평가

XSS	CSRF	SSRF
공격자가 사용자의 브라우저 측에서 악성 스크립트를 실행시켜, 세션 탈취 *사용자의 행위와 관련됨	사용자의 인증 정보를 악용하여 원치 않는 요청을 서버에 전송하도록 유도 *사용자의 행위의 무관함	공격자가 서버 대신 내부 네트워크로 요청을 보내도록 유도하여 내부 시스템 탐색

XSS 웹 앱에서 사용자 입력 값에 대한 필터링이 제대로 이뤄지지 않는 경우, 공격자가 사용자 입력 값을 받는 게시판, URL 등에 악의적인 스크립트(JS) 삽입하여 게시글이나 이메일을 읽는 사용자의 쿠키(세션)를 탈취하여 도용하거나 악성코드 유포 사이트로 Redirect 할 수 있음

CS RF 사용자의 인증 정보 내에서 사용자의 요청을 변조함으로써 해당 사용자의 권한으로 악의적인 공격을 수행할 수 있음

Stored XSS	Reflected XSS	DOM XSS
악성 스크립트가 DB에 영구 저장되며, 피해자가 해당데이터 조회시 실행	악성 스크립트가 요청과 함께 즉시 반영, URL 파라미터 통해 실행	JS DOM 조작 통해 악성 스크립트가 실행

Error based SQL Injection	Union based SQL injection	Blind SQL injection	Stored Procedure based SQL Injection	Mass SQL Injection
SQL 쿼리에 고의로 오류 발생 → 출력되는 에러의 내용으로 필요한 정보 찾아냄. ex) 싱글/더블 쿼테이션	UNION 키워드 사용해 원래 요청에 추가 정보를 얻는 공격 기법. ORDER BY 절, HAVING 이용해 오류 메시지를 통해 컬럼 수 유추. LIKE 뒤 admin의	에러 발생 안되는 사이트에서 DB로부터 특정값 받지 않고, 참/거짓 정보만으로 정보를 얻는 기법 limit, SUBSTR, ASCII 등 사용 1. Boolean SQL	저장 프로시저는 쿼리를 모아 하나의 함수처럼 사용하는 도구. 웹에서 저장 프로시저 접근 권한을 가짐으로써 피해 입힘.	한 번의 공격으로 다량의 DB가 조작해 큰 피해 입힘

Error based SQL Injection	Union based SQL injection	Blind SQL injection	Stored Procedure based SQL Injection	Mass SQL Injection
	adm%을 넣어, 필터링 우회.	injection limit키워드를 통해 하나의 테이블만 조회 → SUBSTR 함수로 첫 글자만 찾게 됨 2. Time based SQL injection 숫자 1을 조작해 현재 사용하고 있는 데이터베이스의 길이를 알아냄 LENGTH 문자열 길이 반환, DATABASE() 사용해 DB이름 반환		

구분	SSO (Single Sign-On)	OAuth (Open Authorization)
주요 목적	여러 서비스에 한 번의 로그인으로 접근할 수 있도록 함 (예: 구글 계정으로 Gmail, YouTube 로그인)	다른 애플리케이션이 **사용자의 자원(데이터, 서비스 등)**에 접근할 수 있도록 권한을 위임함 (예: 페이스북 로그인으로 앱 연동)
역할	인증(Authentication) – 사용자가 누구인지 확인	권한 부여(Authorization) – 특정 리소스에 대한 접근 권한 부여
사용 예시	기업 내부 시스템에서 한 번 로그인하면 모든 사내 서비스에 자동 로그인	외부 앱이 사용자의 구글 드라이브 파일에 접근할 때 사용자 승인 후 액세스

2. Blind SQL 인젝션에서의 활용

Blind SQL 인젝션에서는 `#`를 다음과 같이 사용할 수 있습니다:

- 조건부 쿼리: `#`를 사용해 쿼리의 일부를 조건에 따라 실행되도록 할 수 있습니다.

```
sql                                                                    Copy
SELECT * FROM users WHERE username = 'admin' AND (SELECT CASE WHEN (1=1) THEN sleep(5) ELSE
0 END)#';
```

여기서 `1=1`이 참이면 `sleep(5)`가 실행되고, 거짓이면 `0`이 반환됩니다.

- 에러 기반 Blind SQL 인젝션: `#`를 사용해 에러를 유발하거나 방지할 수 있습니다.

```
sql                                                                    Copy
SELECT * FROM users WHERE username = 'admin' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT((SE
LECT version()), 0x3a, FLOOR(RAND(0)*2)) x FROM information_schema.tables GROUP BY x) y)#';
```

이 쿼리는 데이터베이스 버전을 노출시키려고 시도하며, `#`로 인해 뒤의 구문이 주석 처리됩니다.

- 시간 기반 Blind SQL 인젝션: `#`를 사용해 시간 지연을 유발할 수 있습니다.

```
sql                                                                    Copy
SELECT * FROM users WHERE username = 'admin' AND IF(1=1, sleep(5), 0)#';
```

`1=1`이 참이면 5초간 지연됩니다.